School of Computer Science and Engineering, UNSW

Never Stand Still

WK07-02: Design Patterns for Blockchain Applications

Software Architecture for Blockchain Applications, COMP6452

Salil Kanhere, Nadeem Ahmed

Agenda

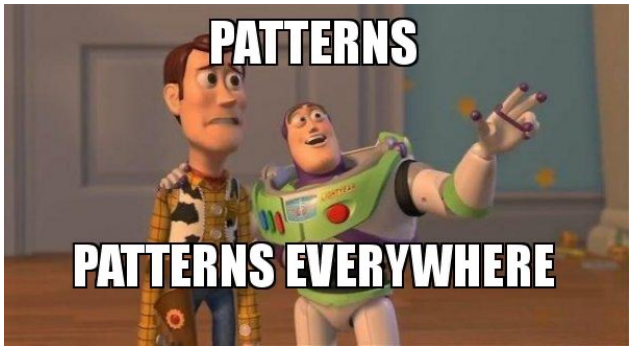
Design patterns

Design patterns for blockchain-based applications

- Pattern collections
- Example patterns

- We'll start with an introduction to what design patterns are.
- Then, we'll discuss what type of pattern collections exists and then discuss a couple of patterns an examples

Design Patterns



Source: https://makeameme.org/

- *"Patterns can help by trying to identify common solutions to recurring problems"*
- *"You can't really understand a pattern without understanding the problem. Problem is essential to help us find a pattern when we need it"*
- *"Patterns are half-baked – you always have to finish them yourself & adapt them to your own solution"*
- *"Patterns are not good or bad – rather, they're either appropriate or not for some situations"*

Source: Martin Fowler, Patterns, IEEE Software, 2013

- Patterns are everywhere, it's just we don't either see them or if we know we still don't know how to use them.
- There are a couple of quotes about patterns.
- Anyway, the more you know the better. However, don't overuse patterns to the point they become useless. Use them if they help to solve a problem… Also, you need to fill missing

Design Patterns

To solve a recurring problem in software development

A description or template for how to solve a problem
- Not a finished design that can be transformed directly into code
- Define constraints that restrict roles of architectural elements & interaction among them
  - Processing, Connectors, Data

Cause trade-offs among quality attributes

- You may have heard about design patterns in your object-oriented programming class.
- In software engineering, a design pattern is a recurring solution to a common problem related to implementation, e.g., factory pattern and proxy.
  - We have seen this diagram before. Design patterns are oriented towards developers (also important to architects too) and they are not application domain specific.
- A design pattern is a description or template for how to solve a problem that can be used in many different situations.
  - It is not a finished design that can be transformed directly into code.
  - A pattern defines constraints that restrict the roles of architectural elements (processing, connectors, and data) and the interaction among those elements.
- Adopting a design pattern causes trade-offs among quality attributes. Therefore, it is important to understand what problem a pattern can solve and the resulting software qualities.

Advantages of Design Patterns

Effective software design requires considering issues that may not become apparent until later stages

- Design patterns document efforts of experts
- Design patterns concern with a flexible software architecture

By reusing patterns, we could prevent subtle errors & enhance code readability

Can speed up development process by providing tested, proven development paradigms

- E.g., Gang of Four (GoF) object-oriented design patterns

- Effective software design requires considering issues that may not become apparent until later stages in the software development life cycle.
  - Whereas design patterns document the efforts of experts who have come across such issues.
  - There is no need to reinvent the wheel.
  - However, they should be documented in a way that is flexible where they can be applied in many software architectures. They also allow flexibility in replacing one pattern with another.
- Therefore, by reusing patterns, we could prevent subtle errors and enhance code readability.
- Can speed up the development process by providing tested, proven development paradigms.
  - E.g., Gang of Four (GoF) object-oriented design patterns are the most widely used design patterns. They were proposed in the book Design Patterns: Elements of reusable object-oriented software

Essential Elements of a Pattern

Pattern Name	• Describe a design problem, its solutions, & consequences in a word or 2
Problem	• Explain problem & its context • Conditions must be met
Solution	• Describe elements that make up design • Relationship, responsibilities, & collaborations
Consequence	• Results & trade-offs of applying pattern • Critical for understanding cost-benefits

- When it comes to documenting a design pattern, there are 4 essential elements.
- The pattern name is a handler we can use to describe a design problem, its solutions, and consequences in a word or 2.
    - The name of a pattern immediately increases our design vocabulary. Having a vocabulary for patterns lets us talk about them with others. But Finding good names has been one of the hardest parts of developing a pattern catalogue.
- The problem describes when to apply the pattern.
    - It explains the problem and the context in which it exists. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
- The solution describes the elements that make up the design, their relationships, responsibilities, and collaborations.
    - The solution doesn't describe a particular concrete design or implementation, because a pattern is like a template that can be applied in many different situations. Instead, the pattern provides an abstract description of a design problem and how a general arrangement of elements (classes and objects in our case) solves it.
- The consequences are the results and trade-offs of applying the pattern.
    - They are critical for evaluating design alternatives and for understanding the costs and benefits of applying the pattern. The consequences of software often concern space, time, and cost trade-offs.

Extended Pattern Form

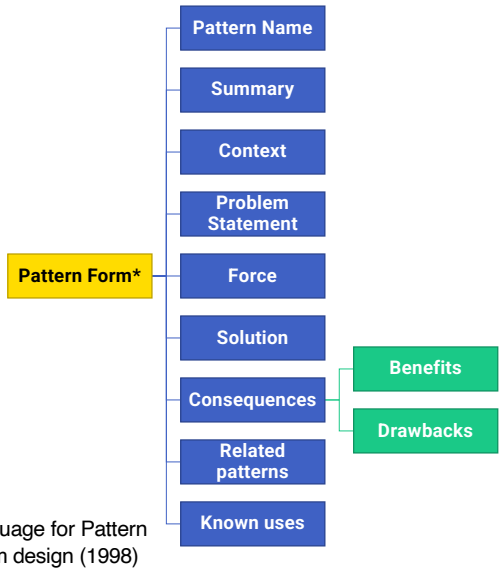
Summary
- 1 - 2 sentence summary of pattern

Context
- Context that the problem exists

Forces
- What makes problem hard?
- Identified with quality attributes

Consequences
- Trade-off between quality attributes

Related patterns
- Other patterns that provide alternative solutions or work together



* Meszaros, G., et al.: A Pattern Language for Pattern Writing. Pattern languages of program design (1998)

- This is called the extended pattern form which is usually used when documenting patterns in academic publications. Even industry publications are not very far from this.
- This is the structure of the pattern form or template we use to describe the blockchain application patterns.
- This extended pattern form has a few additional elements like
  - A summary that summarises the pattern in 1 or to sentences.
  - In the context that the problem exists, e.g., in blockchain-based applications, activities might need to be authorised by multiple blockchain addresses.
  - Forces are an explicit discussion of the forces which make the problem difficult to solve, e.g., lost keys, and cost of transactions. Forces are identified with the corresponding quality attributes; the solution will propose a trade-off between them.
  - Consequents are trade-offs between quality attributes that are affected by the pattern, e.g., while security can be enhanced with multi-signature transactions more transactions mean more cost.
    - Sometimes consequences are presented separately as benefits and drawbacks.
  - Related patterns –Other patterns that provide alternative solutions or can work together.
  - Some examples of real-world known uses of the pattern.
- Regarding the consequences, we distinguish the benefits and drawbacks.

Agenda

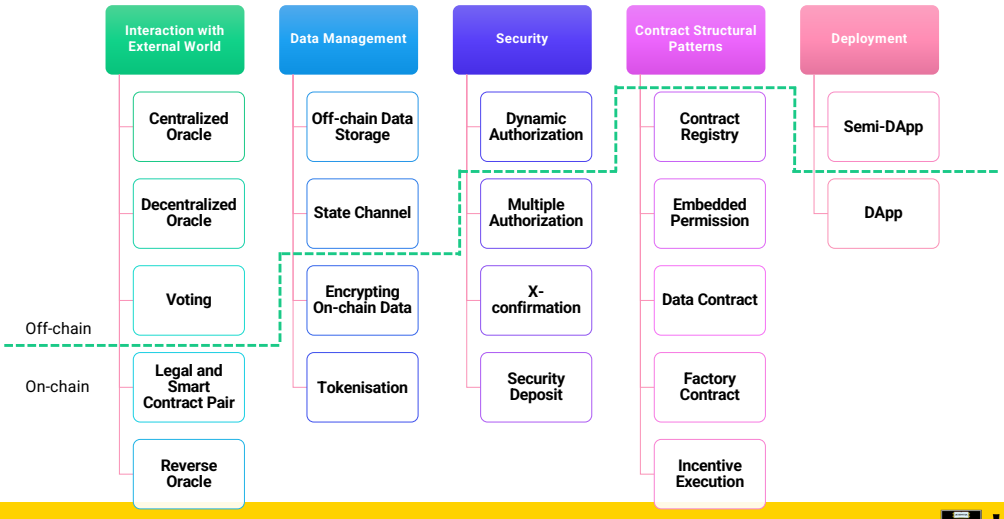
Design patterns

Design patterns for blockchain-based applications

- Pattern collections
- Example patterns

- We'll start with an introduction to what design patterns are.
- Then, we'll discuss what type of pattern collections exists and then discuss a couple of patterns an examples

Blockchain-based Application Pattern Collection



- Here is a blockchain-based application pattern collection that presents patterns under several categories. Also, the scattered line shows whether the pattern sits on-chain or off-chain.
- Interaction with External World patterns focuses on on-chain and off-chain data flow.
  - It covers 3 types of oracles we discussed so far,
  - The voting pattern has the usual meaning and can be used to vote for data reported by decentralised oracles.
  - Legal and smart contract pair can be used to establish a bidirectional binding between a legal agreement and a corresponding smart contract.
- Data management patterns focus on managing data on and off the blockchain.
  - We have already talked about options like storing data off-chain and encrypting data before storing on-chain.
  - The tokenisation pattern is about the idea of using tokens to represent an asset.
  - State channel patterns can be used to conduct low-value or high-velocity transactions off-chain. We'll discuss this pattern in a later lecture.
- Security aspects like authentication based on digital signatures are core components of blockchain design.
  - We already discussed multi-signature which is also known as multiple-authorisation.
  - Dynamic Authorisation uses a secret created off-chain to bind the authority for a transaction dynamically.
  - X-Confirmation waits for a sufficient number of blocks as confirmations to ensure that a transaction added into the blockchain is immutable with high probability, e.g., waiting for 6 blocks in Bitcoin.
  - The security Deposit pattern is about a user setting aside a certain amount of money as a security deposit.
- Contract Structural Patterns focus on smart contract design structures.
  - We'll talk about some of these patterns later.
- Deployment patterns focus on the deployment of blockchain-based applications.
- See https://research.csiro.au/blockchainpatterns/ for a large collection of patterns.

Application Pattern Collection - Summary

Interaction with External World

Centralised Oracle
- Introducing external state into blockchain environment through an oracle

Decentralised Oracles
- Introducing external state into blockchain environment through multiple oracles

Voting
- A method for a group of blockchain users to make a collective decision

Reverse Oracle
- Introducing blockchain state into external world through an oracle

Legal & smart contract pair
- A bidirectional binding between a legal agreement & corresponding SC that codifies legal agreement

Data Management

Encrypting On-chain Data
- Ensuring confidentiality of data stored on blockchain by encrypting it

Tokenisation
- Using tokens on blockchain to represent transferable digital or physical assets

Off-chain Data Storage
- Using hashing to ensure integrity of arbitrarily large datasets which may not fit directly on blockchain

State Channel
- TXs that are too small in value or require much shorter latency, are performed off-chain with periodic recording of net TX settlements on-chain

Security

Multiple Authorisation
- TXs are required to be authorised by a subset of pre-defined addresses

Dynamic Authorisation
- Using a hash created off-chain to dynamically bind authority for a TX

X-Confirmation
- Waiting for enough number of blocks as confirmation to ensure that a TX added into blockchain is immutable with high probability

Security Deposit
- A deposit from a user, which will be paid back to user for her honesty or given to others to compensate them for dishonesty of user

Here's a 1 sentence summary of those patterns.

Application Pattern Collection – Summary (Cont.)

Structural Patterns of Contract

Contract Registry
- Address and version of a SC is stored in a contract registry

Embedded Permission
- Restrict access to invocation of functions defined in a SC

Data Contract
- Storing data in a separate SC from business logic

Factory Contract
- An on-chain template contract used as a factory that generates SC instances from template

Incentive Execution
- A reward to caller of a SC function for invocation

Deployment

DApp
- Blockchain-based application hosted on P2P network, with a website that allows users to interact with SCs

Semi-DApp
- Blockchain-based application with a website that can be browsed using a conventional web browser without any ADApp plugin

Here's a 1 sentence summary of those patterns.

Smart Contract Patterns

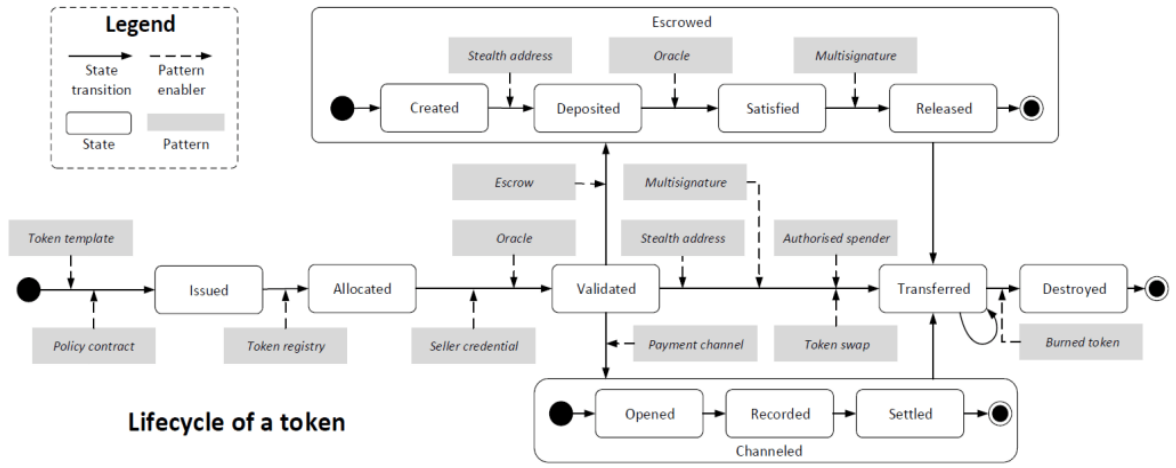
BC Pattern Name	SC Pattern Name
Dynamic Authorization	Commit and Reveal
Centralized/Decentralized Oracle	Oracle (Data Provider)
Data Contract	Data Segregation
Contract Registry (within a contract)	Satellite
Contract Registry	Contract Register

Maximilian Wöhrer and Uwe Zdun, "Design patterns for smart contracts in the Ethereum ecosystem," IEEE iThings, GreenCom, CPSCom, and SmartData. 2018.

Category	Pattern	Example Contract
Action and Control	Pull Payment	Cryptopunks
	State Machine	DutchAuction
	Commit and Reveal	ENS Registrar
	Oracle (Data Provider)	Etheroll
Authorization	Ownership	Ethereum Lottery
	Accesss Restriction	Etheroll
Lifecycle	Mortal	GTA Token
	Automatic Deprecation	Polkadot
Maintenance	Data Segregation	SAN Token
	Satellite	LATP Token
	Contract Register	Tether Token
	Contract Relay	Numeraire
Security[3]	Checks-Effects-Interaction	CryptoKitties
	Emergency Stop	Augur/REP
	Speed Bump	TheDAO
	Rate Limit	etherep
	Mutex	Ventana Token
	Balance Limit	CATToken

- Here's another pattern collection focusing on smart contracts.
- Action and Control is a group of patterns that provide mechanisms for typical operational tasks of a contract like handling payments and maintaining the internal state.
- Authorisation is a group of patterns that control access to smart contract functions and provide basic authorisation.
- A lifecycle is a group of patterns that control the creation and destruction of smart contracts.
- Maintenance is a group of patterns that provide mechanisms for active contracts within the limits of immutability.
- Then there is a bunch of security-related patterns.
- It is also worthwhile noting that the same pattern may be known under different names depending on who drafts them. For example,
    - A data contract is also known as data segregation as it segregates data from the business logic.
    - Oracle is one pattern in one collection, while in the other it is split as a centralised and decentralised oracle. So a pattern may be documented at different levels of abstraction.

Token Patterns



See https://research.csiro.au/blockchainpatterns/

Here's set of token pattern that are given in grey rectangles.
Note how the patterns are presented based on the lifecycle of a token from issuing a token to eventually destroying it.

Factory Contract Pattern

Summary

- Use an on-chain contract as a factory to spawn contract instances from a SC template

Context

- Blockchain application needs to use multiple SCs with similar functionality but different data
- Contract instance is created by instantiating a contract template
- Stored off-chain in a code repository

Problem

- How can we ensure all SC instances follow the same contract template?

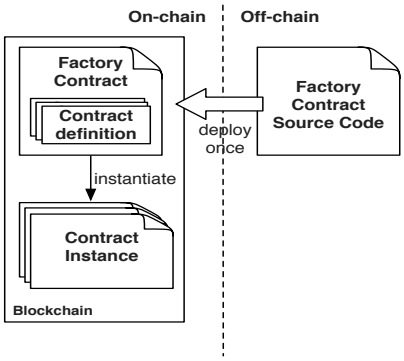
Forces

- Integrity – Off-chain contract template can't guarantee consistency between different SC instances
- Dependency management – Storing SC off-chain introduces more components
- Deployment – Extra effort is needed to deploy a SC from off-chain source code
- Secure code sharing – Blockchain is a secure platform for sharing contract code

- Perhaps you may have heard of the factory pattern in object-oriented programming. It is a design pattern that provides an interface for creating objects in a super class while allowing subclasses to alter the type of objects that will be created.
- The same idea can be applied to deploy, multiple smart contract instances from another contract, while changing some of their initialisation attributes.
- Here we document the pattern in extended form.

- Summary: A template smart contract deployed on-chain is used as a contract factory to deploy contract instances from the template.
- Context: An application based on a blockchain needs to use multiple instances of a standard contract with customisation.
  - Each contract instance is created by instantiating a contract template. E.g., in a business process management system, each of the business process instances might be represented by a smart contract being generated from a contract template representing the business process model.
  - The template is stored off-chain in a code repository.
- Problem: How can we ensure all SC instances follow the same contract template? Also, Keeping the contract template off-chain cannot guarantee consistency between different smart contract instances created from the same template because the source code of the template can be independently modified.
- Forces
  - It is difficult to protect the integrity of the off-chain code. As opposed to a traditional code repository, code changes deployed on a smart contract can be strictly limited or prohibited.
  - Dependency management. Storing the source code of smart contracts off-chain in a code repository introduces the issue of integrating more systems into the blockchain-based application.
  - Secure code sharing. Blockchain can be used as a secure platform to share the code of smart contracts.
  - Deployment. If a public code repository, like GitHub, is used to store the source code of a smart contract, a component is needed to implement the function of deploying a smart contract on the blockchain, otherwise, the end users need to understand how to deploy smart contracts by sending transactions with the customized source code of the contract definition.

Factory Contract Pattern – Cont.



Solution

- Factory contract embedding template contract is deployed once from off-chain source code
- SC instances are generated by passing parameters to factory contract to deploy customised instances

Consequences

- Integrity – On-chain factory contract guarantees consistency
- Efficiency – SC instances are generated by calling a function
- Cost – TX fees on public blockchain to deploy & invoke factory contract

Related Patterns

- Contract registry

Known uses

- Tutorial from Ethereum developer
- Applied in a real-world blockchain-based health care application
- Business process management system using factory to spawn process instances

- Solution: As seen from the figure deploy a factory contract embedding the contract instances to be deployed. Then invoke a function on that contract with desired input parameters to spawn a new contract instance.
  - The factory contract is deployed once from the off-chain source code.
  - The factory may contain the definition of multiple smart contracts.
  - Smart contract instances are generated by passing parameters to the contract factory to instantiate customised smart contract instances.
  - Factory contract is analogous to a class in an object-oriented programming language. Every transaction that generates a smart contract instance essentially instantiates an object of the factory contract class.
  - This contract instance (the object) will maintain its properties independently of the other instances but with a structure consistent with its original template.
- Consequences
  - Benefits
    - Security. Keeping the factory contract on-chain guarantees the consistency of the contract definition.
    - Efficiency. If the contract definition is kept on-chain in a factory contract, smart contract instances are generated by calling a function defined in the factory contract.
  - Drawbacks:
    - Deployment cost. If a public blockchain is used, using a factory contract requires the extra cost to deploy the factory contract.
    - Function call cost. If a public blockchain is used, creating a new smart contract instance requires the extra cost to call a function defined in the factory contract.
- Related patterns – This pattern is related to another pattern called contract registry that keeps track of deployed contract names and their addresses.
- Known use
  - A tutorial from Ethereum developers about how to create a contract factory from which smart contract instances can be created.
  - Factory pattern has been applied in a real-world blockchain-based healthcare application.
  - The business process management system in academic work uses a contract factory to generate process instances.

Contract Registry Pattern

Summary

- Before invoking a SC, lookup contract name to find address of the latest contract instance

Context

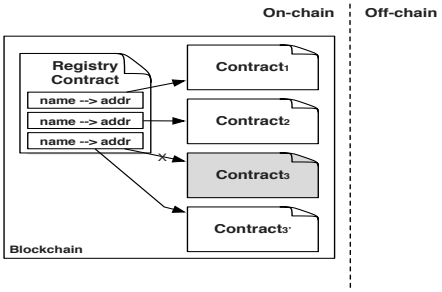
- Blockchain-based applications need to be upgraded to new versions
  - Fix bugs or fulfil new requirements

Problem

- How to reach the latest version of smart contracts?

Forces

- Immutability – On-chain SC code is immutable
- Upgradability – Fundamental need to upgrade SCs
- Human-readable identifier – SC addresses aren't human-readable



- Here's another pattern that is related to the factory pattern.
- Summary: Maintain a registry mapping a smart contract's name and the address of its latest version as a *(name, address)* pair. Before invoking a smart contract, look up the registry to find its address.
- Context: Like any software application, blockchain-based applications need to be upgraded to new versions. To do so, the on-chain functions defined in smart contracts need to be updated to fix bugs and fulfil new requirements.
- Problem: A smart contract deployed on a blockchain cannot be upgraded because the smart contract (binary) code is stored as immutable ledger data. Therefore, when a new version of the smart contract is deployed, it will be assigned a different address. Not all transaction issuers may be aware of the existence of the new smart contract version or its address. How to reach the latest version of smart contracts?
- Forces: The problem requires balancing the following forces:
  - Immutability. Every bit of data, including deployed smart contracts, stored on the blockchain is immutable.
  - Upgradability. There is a fundamental need to upgrade all but short-lived applications and their smart contracts over time.
  - Human-readable contract identifier. The identifier of a smart contract on blockchain platforms, like Ethereum, is a hexadecimal address, which is not human-readable.
- The figure illustrates the idea of having a registry of smart contracts. The registry keeps track of the (name, address) pair. Then before invoking a smart contract, the user needs to query the registry contract to find the address of the latest smart contract instance. When a smart contract is upgraded by deploying a new instance, the address on the registry is updated to the point of the new instance.

Contract Registry Pattern – Cont.

Solution

- On-chain registry contract maintains contract (name, addresses) pairs
- Registry contract address is well-known
- User queries address of the latest version of SC from registry
- Upgrade contract by replacing old contract address with new contract address

Consequences

- Human-readable & constant contract names
- Transparent upgradability
- Look-up based on name & version
- Modifying interfaces is challenging if used by external components
- Cost of maintaining registry

Related Patterns

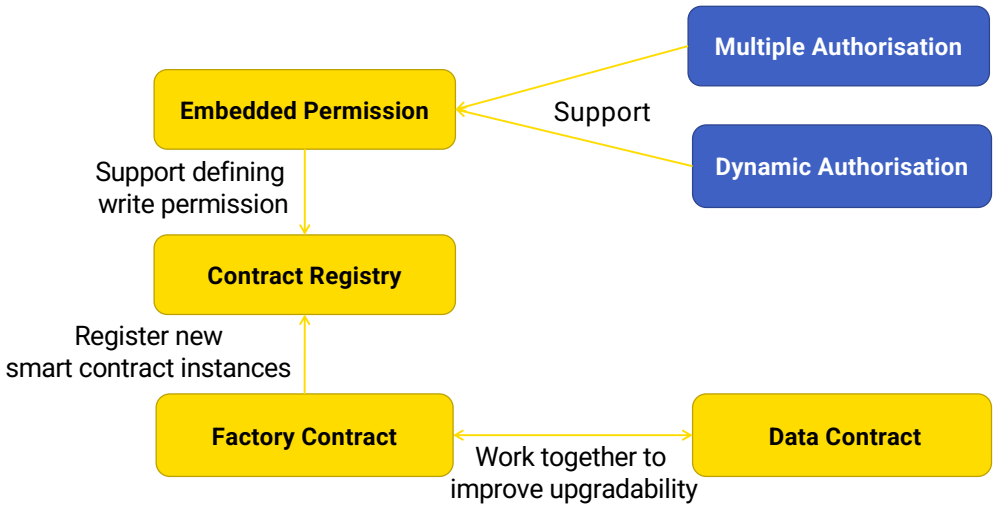
- Data Contract
- Embedded Permission

Known uses

- ENS (Ethereum Name Service)
- Regis - In-browser application for registries deployment & management

- Solution: An on-chain registry contract is used to maintain a mapping between user-defined symbolic names and the blockchain addresses of the registered contracts.
  - The address of the registry contract needs to be advertised off-chain. The creator of a contract can register the name and the address of the new contract to the registry contract after the new contract is deployed.
  - The invoker of a registered contract retrieves the latest version of the new smart contract from the registry contract.
  - The corresponding functions provided by the registered contract can be upgraded by replacing the address of the old version contract in the registry contract with the address of a new version without breaking the dependency between the upgraded smart contract and other smart contracts that depend on its functions.
  - The address of a contract is stored as a variable in the registry contract. The value of contract variables can be updated. The registry contract can have a permission control module to maintain the writing permission. Note that all the previous values of the variable are still stored on the blockchain.
- Consequences:
  - Benefits
    - Human-readable contract name. The registry contract maintains a mapping between human-readable names and the hexadecimal addresses of the smart contracts.
    - A human-readable form of smart contract names is desired, for example, to be exposed to the user interface. A human-readable name is also useful for developers.
    - Constant contract name. The smart contract associated with a registered name can be updated without changing its name. This way dependencies relying on the name of the smart contract do not get broken.
    - Transparent upgradability. The smart contract associated with a registered name could be replaced by a new version without breaking the dependencies based on the human-readable name.
    - Version control. Version control can be integrated into the registry contract as well to allow a lookup based on the name and version of a smart contract. Old versions of a smart contract that are no longer needed should be terminated.
  - Drawbacks:
    - Limited upgradability. Upgradability is still limited if the functions defined in the smart contract are called by other contracts. Although the implementation of the function can be upgraded, the interface (that is function signature) cannot be modified without breaking the link to dependent smart contracts.
    - Similar methods for API / service interface management need to be implemented, e.g. through versioning and depreciation flags.
    - Cost. There is an additional cost to maintain a registry that contains the mapping between the contract names and their addresses.
    - Furthermore, all the inter-contract function calls require a registry lookup to find the latest version of the smart contract to be invoked.
- Related patterns:
  - A data contract pattern is used to separate data from business logic. That pattern can use the contract registry pattern to keep track of the logic contract address.
  - Embedded permissions patterns provide a mechanism to define permissions to update addresses maintained by the registry.
- Known use
  - ENS is a name service on the Ethereum blockchain, which is implemented as a smart contract.
  - ENS maintains a mapping between both smart contracts on-chain and resources off-chain and simple, human-readable names.
  - ENS can be viewed as a contract registry built in a blockchain platform, which is accessible to everyone. A blockchain-based application can also maintain a separate registry contract for the application.
  - Regis is an in-browser application that makes it easy to build, deploy and manage registries as smart contracts on the Ethereum blockchain. It allows user-defined key-value pairs. It can be used to create a contract registry.

Related Patterns



- This diagram illustrated how a set of patterns are related to each other. Such information is important in understanding related and alternative patterns.
- Contract registry provides a registry for smart contract.
- Factory contract create a new contract instance, and registers the new instance to the contract registry.
- Embedded permission add permission to contract registry.
- Data contract works with the created smart contract to improve upgradability.
- Multiple authorisation and dynamic authorization can support the permission control.

Contract Relay (aka Proxy)

Summary
- Use a proxy contract to relay TXs

Context
- SCs need to be upgradable

Problem
- How to access latest version of a SC?

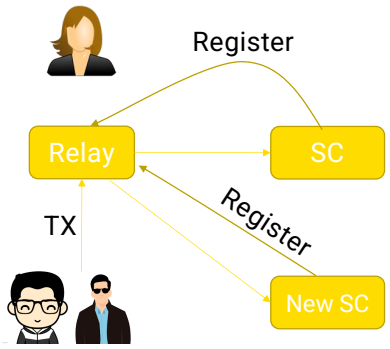
Forces
- Immutability – On-chain SC code is immutable
- Upgradability – Fundamental need to upgrade SCs
- Cost of redeployment & state data

Solution
- Define a proxy contract
- SC users always interact with the proxy contract
- Proxy relays all TXs to the most recent contract version

Consequences
- Immutability isn't affected as SC isn't modified
- Latest contact can still be reached
- Calls & delegated calls are costly

- Let us look at another contract that provides an alternative solution to the contract registry pattern.
- This patterns belongs to the smart contract maintenance patterns group.
- It is used to relay a smart contract call to the latest version of the contract.

Contract Relay – Cont.




```
pragma solidity ^0.4.17;
import "../authorization/Ownership.sol";
contract Relay is Owned {
  address public currentVersion;

  function Relay(address initAddr) public {
    currentVersion = initAddr;
    owner = msg.sender;
  }

  function changeContract(address newVersion) public
      onlyOwner() {
    currentVersion = newVersion;
  }

  // fallback function
  function() public {
    require(currentVersion.delegatecall(msg.data));
  }
}
```


onlyOwner modifier is inherited from Owned contract
Fallback function is like default in switch-case statements
What's the difference between Registry Contract & Contract Relay?

Pull Payment

Summary

- Let receiver of a payment withdraw funds from sender

Context

- SC wants to transfer funds to another address
  - address.send(value) – 2,300 gas limit
  - address.transfer(value) – 2,300 gas limit
  - address.call.value(value) – all available gas or set limit
- Send operation can fail when it calls *fallback* function on receiver

Problem

- When a SC calls another SC, it hands over control to that SC
- How to prevent callee SC from re-entering called SC & trying to manipulate its state or hijack control?

Forces

- It's risky to handover control to contracts that are not under your control
  - E.g., re-entrancy attack
- Who pay TX fee?

- This pattern promotes the idea of getting a recipient to pull cryptocurrency than pushing it as it could lead to a couple of issues such as either the recipient leading to a deadlock or may be able to withdraw more crypto than they are supposed to.
- If the payment recipient is a SC, calling theses methods triggers execution of a so-called fallback function in the receiver.
  - Fallback function is a name and parameterless function, that is called when the function signature doesn't match any of the available functions in a Solidity contract.
  - This can be exploited to call the called contract back
- When a smart contract wants to transfer funds to another address there are a couple of options:
  - address.send send only false when failed
  - address.transfer throw an exception when failed
  - address.call.value send only false when failed. But can set a max gas limit
  - Note that 2,300 gas is currently only enough to log an event.
- [See https://solidity.readthedocs.io/en/v0.6.10/units-and-global-variables.html#address-related](https://solidity.readthedocs.io/en/v0.6.10/units-and-global-variables.html#address-related)

Pull Payment – Cont.

```
pragma solidity ^0.4.17;
contract Auction {
  address public highestBidder;
  uint highestBid;

  function bid() public payable {
    require(msg.value >= highestBid);
    if (highestBidder != 0) {
      // if call fails causing a rollback,
      // no one else can bid
      highestBidder.transfer(highestBid);
    }
    highestBidder = msg.sender;
    highestBid = msg.value;
  }
}
```

```
pragma solidity ^0.4.17;
contract Auction {
  address public highestBidder;
  uint highestBid;
  mapping(address => uint) refunds;

  function bid() public payable {
    require(msg.value >= highestBid);
    if (highestBidder != 0) {
      // record the underlying bid to be refund
      refunds[highestBidder] += highestBid;
    }
    highestBidder = msg.sender;
    highestBid = msg.value;
  }

  function withdrawRefund() public {
    uint refund = refunds[msg.sender];
    refunds[msg.sender] = 0;
    msg.sender.transfer(refund);
  }
}
```


- If the transfer() fails, the function will revoke. This mean the current highest bigger can remain so even if someone is offering a higher bid.
- Mitigates this problem by isolating the external call into its own transaction that can be initiated by the recipient of the call.
- How would the previous highest bidder know that it's no longer the highest bidder and need to pull its bid?

Pull Payment – Cont.

Solution

- Let receiver of a payment withdraw funds
- Track funds to be released separately & provide a withdraw funds
- Make sure to update state before calling *transfer*()

Consequences

- Enhanced security
- Convivence
  - E.g., previous highest bidder not explicitly informed
- More complex code